

Query Processing and Optimization

Slides credit goes to

Mr. Said Nabi, FCSE

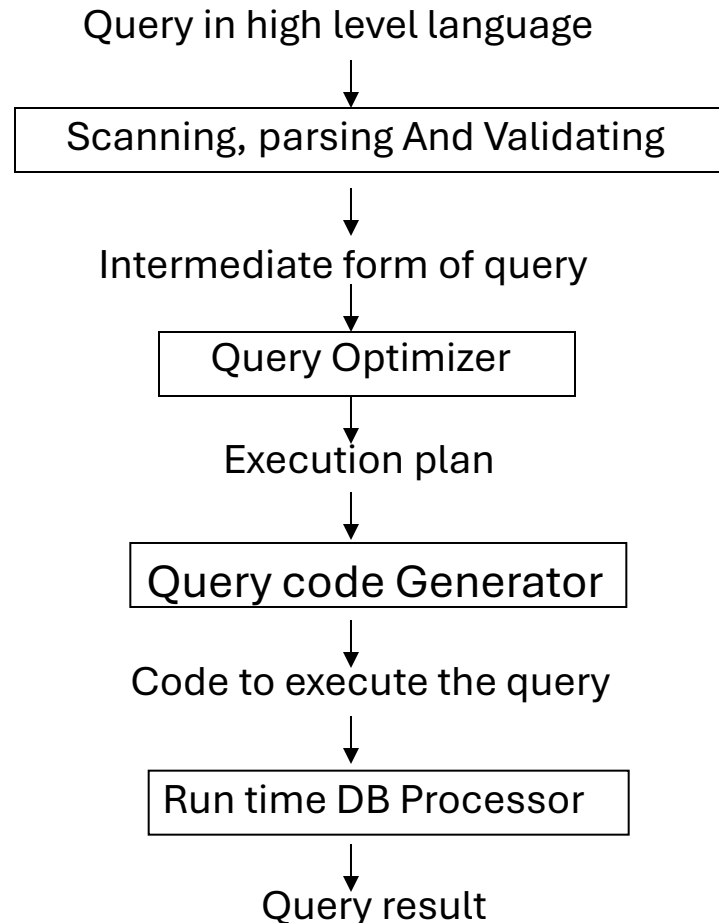
Objectives

- Introduction
- Major Phases in Query Processing
- Optimizing Queries

Introduction

- With declarative languages such as SQL, the user specifies what data is required rather than how it is retrieved
- Relieves user of knowing what constitutes a good execution strategy
- Also gives the DBMS more control over system performance.

Processing a High-Level Query



Processing a High-Level Query

- The **scanner** identifies the query tokens—such as SQL keywords, attribute names, and relation names—that appear in the text of the query, whereas the
- **parser** checks the query syntax to determine whether it is formulated according to the syntax rules (rules of grammar) of the query language.
- The query must also be **validated** by checking that all attribute and relation names are valid and semantically meaningful names in the schema of the particular database being queried.

Processing a High-Level Query

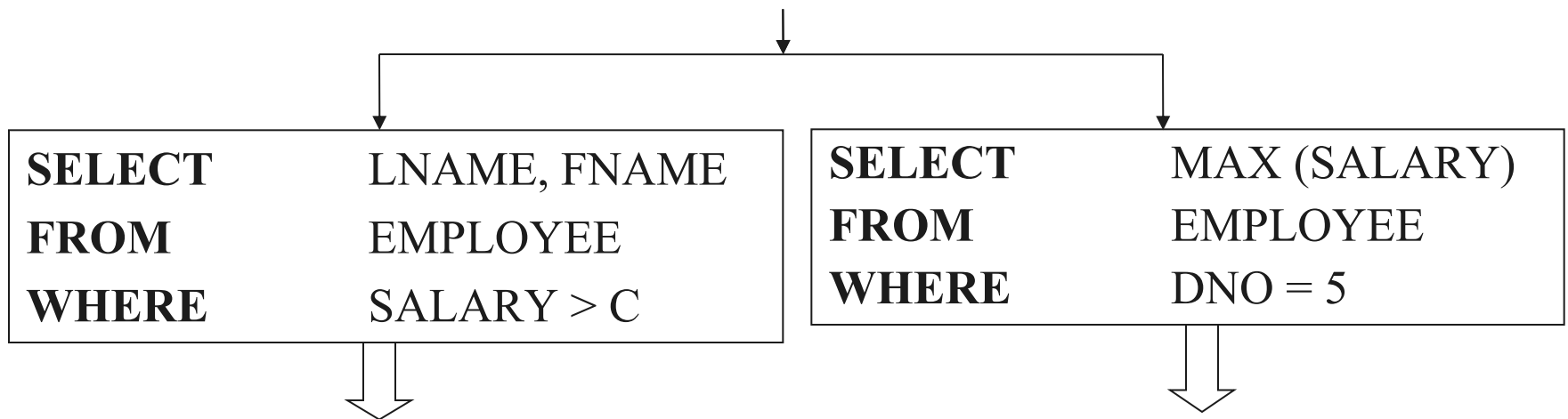
- An internal representation of the query is then created, usually as a tree data structure called a **query tree**.
- It is also possible to represent the query using a graph data structure called a **query graph**, which is generally a **directed acyclic graph (DAG)**.
- The DBMS must then devise an **execution strategy** or **query plan** for retrieving the results of the query from the database files.
- A query has many possible execution strategies, and the process

Major Phases in Query Processing

- **Scan, Parse, and Validate Query:** Transform query written in high level language (e g. SQL), into correct efficient execution strategy expressed in low-level language (e. g. Relational Algebra)
- **Optimize query and generate code:** produce an execution strategy or plan and run-time code.
- **Execute query:** Implement the plan to retrieve required data.

Example: Translating SQL Queries into Relational Algebra

```
SELECT      LNAME, FNAME
FROM        EMPLOYEE
WHERE       SALARY > (  SELECT      MAX (SALARY)
                        FROM        EMPLOYEE
                        WHERE       DNO = 5);
```



Example: Translating SQL Queries into Relational Algebra

SELECT	LNAME, FNAME	SELECT	MAX (SALARY)
FROM	EMPLOYEE	FROM	EMPLOYEE
WHERE	SALARY > (	WHERE	DNO = 5);

↓

SELECT	LNAME, FNAME
FROM	EMPLOYEE
WHERE	SALARY > C

⇓

$$\pi_{\text{LNAME, FNAME}} (\sigma_{\text{SALARY} > C} (\text{EMPLOYEE}))$$

↓

SELECT	MAX (SALARY)
FROM	EMPLOYEE
WHERE	DNO = 5

⇓

$$\mathcal{F}_{\text{MAX SALARY}} (\sigma_{\text{DNO}=5} (\text{EMPLOYEE}))$$

Optimizing Queries

- The process of choosing a suitable execution strategy for processing a query
- As there are many equivalent transformations from same high-level query, aim is to choose the one that minimizes resource usage
- Generally, an efficient execution plan reduces the total execution time of a query; thereby reducing the response time of a query
- The problem is computationally intractable with large number of relations, so the strategy adopted is reduced to finding a near optimum solution
- Two main techniques for query optimization are:
 - **Heuristic rules that order operations in a query**
 - **Comparing different strategies based on relative cost, and selecting one that minimizes resource usage.**

Steps in Query Optimization

- **Process for heuristics optimization**
 1. The parser of a high-level query generates an initial internal representation;
 2. Apply heuristics rules to optimize the internal representation.
 3. A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.
- **The main heuristic is to apply first the operations that reduce the size of intermediate results.**
 - **E.g., Apply SELECT and PROJECT operations before applying the JOIN or other binary operations.**

Query Trees

- A tree data structure that corresponds to a relational algebra expression.
- It represents the input relations of the query as **leaf nodes** of the **tree**, and represents the relational algebra operations as **internal nodes**.
- An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation.
- **The order of execution of operations starts at the leaf nodes**, which represents the input database relations for the query, and *ends at the root node*, which represents the final operation of the query. **The execution terminates when the root node operation is executed and produces the result relation for the query.**

Query graph:

- A graph data structure that corresponds to a relational calculus expression.
- It does *not* indicate an order on which operations to perform first.
- There is only a *single* graph corresponding to each query.

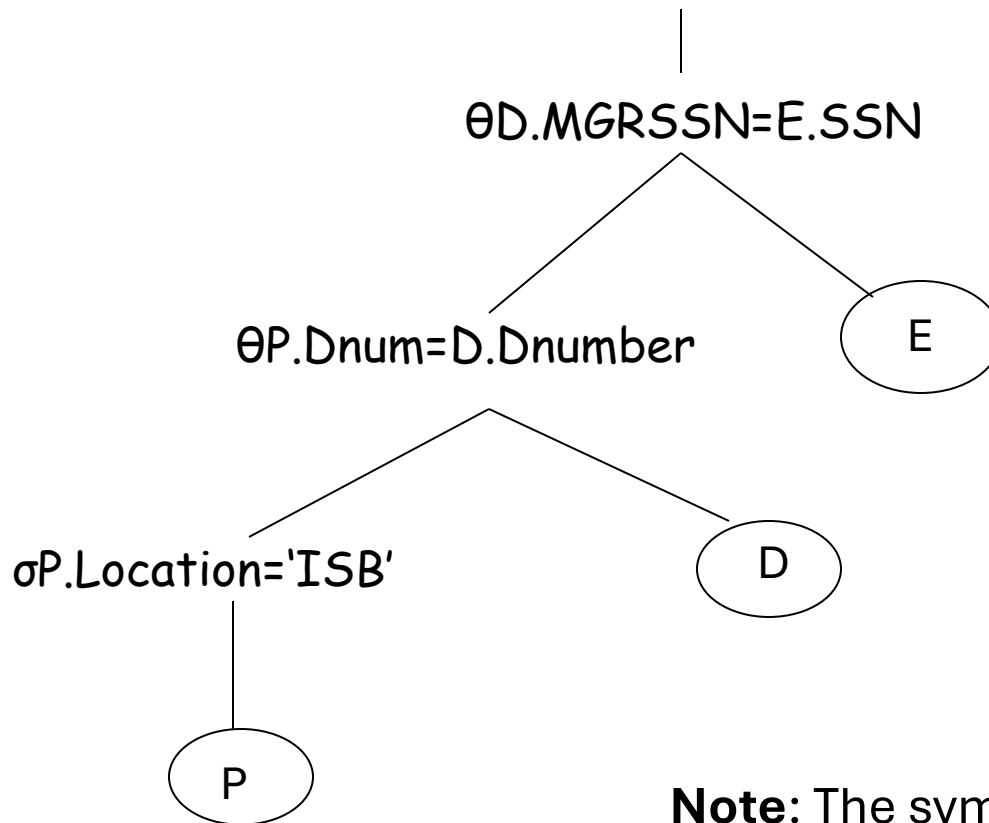
Query Tree Example

- For every project located in 'ISB', list the project number, the controlling department number and the department manager's last name, address, and birth date.

```
SELECT Pnumber, Dnum, Lname, Address, Bdate  
FROM Project, Department, Employee  
WHERE Dnum=Dnumber  
AND MgrSSN = SSN  
AND Plocation = 'ISB'
```

Query Tree: Example

$\Pi P.Pnumber, P.Dnum, E.Lname, E.Address, E.Bdate$

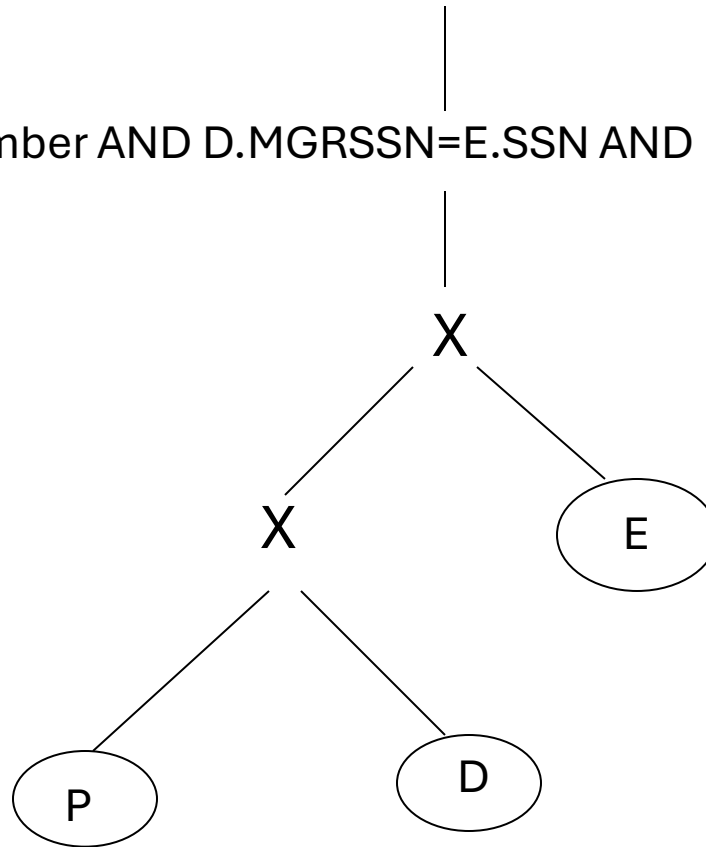


Note: The symbol θ represents JOIN

... --- Query Tree: Example

Π P.Pnumber, P.Dnum, E.Lname, E.Address, E.Bdate

σ P.Dnum=D.Dnumber AND D.MGRSSN=E.SSN AND P.Location='ISB'

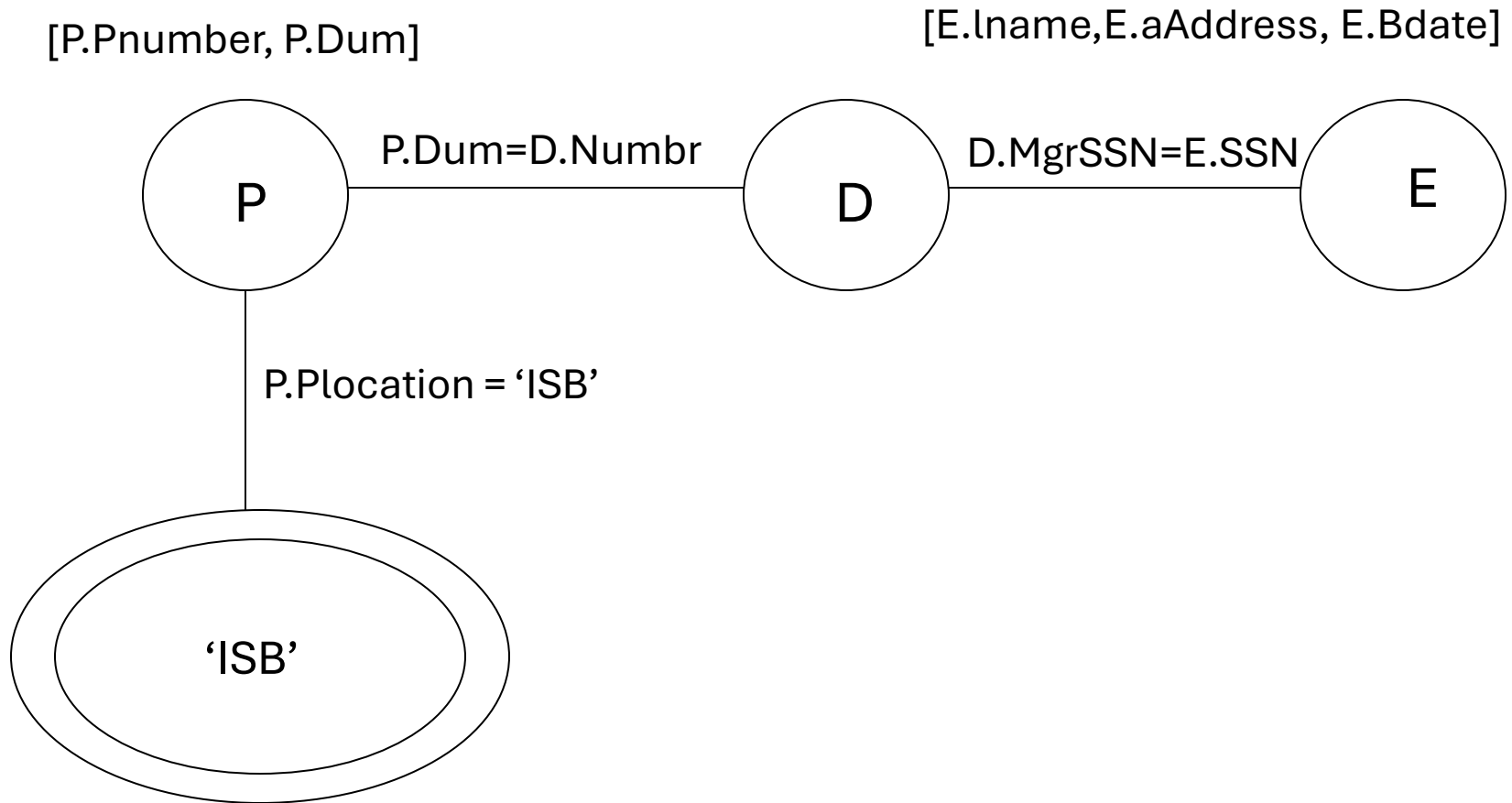


--- Query Graph Example ...

- For every project located in 'ISB', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
SELECT Pnumber, Dnum, Lname, Address,  
       Bdate  
FROM Project, Department, Employee  
WHERE Dnum = Dnumber  
AND MgrSSN = SSN  
AND Plocation = 'ISB'
```

... --- Query Graph Example



ERD & Relational Algebra Pipeline

So far we know the process is as following:

Software Requirements → ER diagram → mapping relational model → SQL

Relational Algebra

- A set of operations for the relational model.
 - Based on sets (no duplicate tuples)
- Enable a user to specify basic retrieval requests
- The algebra operations produce new relations.
 - The result of a retrieval is a new relation.
- A sequence of relational algebra operations forms a relational algebra expression
 - Result
 - A relation.
 - Represents the result of a database query.

We know the topics on relational algebra

- Select
- Project
- Union
- Intersection
- Minus
- Cartesian product
- Join
- Natural join

Relational Algebra (Select Operation)

Cascading / Commutative

The SELECT Operation

- $\sigma_{\langle \text{selection condition} \rangle}(R)$
 - See which table we want to query.
- Filter - only those tuples that satisfy a qualifying condition appear in the result.
- Result: *subset* of the tuples
- The $\leftarrow$ symbol is an assignment operator
- Commutative
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(R)) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R))$
 - Last example
 - Small example:
 - $\sigma_{\text{dno}=5}(\sigma_{\text{salary}>30000}(\text{Employee}))$ is the same as:
 - $\sigma_{\text{salary}>30000}(\sigma_{\text{dno}=5}(\text{Employee}))$
 - Or.... cascade it....
- A cascaded SELECT operation *may be applied in any order*
 - $\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition3} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R)))$
 - Small example continued:
 - ... $\sigma_{\langle \text{dno} = 5 \rangle \text{ and } \langle \text{salary} > 30000 \rangle}(\text{Employee})$

- Relational Algebra Transformation

- The following list gives a basic selection of relational algebra transformation:

- **Cascade of selection:** Conjunctive selection operations can cascade into individual selection operations

$$\sigma p \text{ AND } q \text{ AND } r (R) = \sigma p (\sigma q (\sigma r (R)))$$

- **Cascade of projection:** In a sequence of projection operations, only the last in sequence is required.

$$\Pi L(\Pi M(\dots(\Pi N(R)) = \Pi L(R)$$

Relational Algebra Transformation

Cascade of selection

The expression $\sigma_{p \text{ AND } q \text{ AND } r}(R) = \sigma_p(\sigma_q(\sigma_r(R)))$ represents the **Cascade of Selection** rule in relational algebra ¹. It is a fundamental transformation used by a DBMS to optimize how a query is executed.

Here is a breakdown of what this expression means and why it is important:

1. Breaking Down the Symbols

- σ (**Sigma**): This represents the **selection** operation, which filters rows (tuples) from a table based on specific conditions ¹ ².
- R : This is the **input relation** (the table) being queried ¹.
- p, q, r : These are the **selection predicates** or conditions (e.g., `Salary > 5000` or `DNO = 5`) ¹ ².

Relational Algebra Transformation

(Cascade of Selection)

2. What the Expression States

- **The Left Side** ($\sigma_{p \text{ AND } q \text{ AND } r}(R)$): This is a **conjunctive selection**. It means the system tries to find all rows in table R that satisfy conditions p , q , and r all at once ¹.
- **The Right Side** ($\sigma_p(\sigma_q(\sigma_r(R)))$): This is a **cascaded sequence** of selections. It tells the system to first filter the table for condition r , then filter that smaller result for condition q , and finally filter that even smaller result for condition p ¹.
- **The Equals Sign (=)**: This indicates that both methods are **equivalent**—they will always produce the exact same final result set ¹ ³.

Relational Algebra Transformation

(Cascade of Selection)

3. Why This Is Used in Optimization

While both sides produce the same result, the right side is often more efficient for the DBMS to execute because of **Heuristic Optimization** 4 5 :

- **Reducing Intermediate Results:** The main heuristic rule is to apply operations that reduce the size of data as early as possible 4 . By cascading selections, the DBMS can "push down" specific filters toward the leaf nodes (the original tables) of a query tree 5 6 .
- **Early Filtering:** Instead of waiting to apply a complex filter after a large operation like a Join or Cartesian product, the system can apply individual filters (p , q , or r) immediately 5 6 . This ensures that fewer rows are passed up the tree to the more "expensive" operations, saving time and memory 3 4 .
- **Ordering for Speed:** Because selection is also **commutative** ($\sigma_q(\sigma_p(R)) = \sigma_p(\sigma_q(R))$), the DBMS can choose to apply the most restrictive condition first (the one that removes the most rows) to maximize efficiency 5 7 .

Relational Algebra Transformation

(Cascade of Projection)

The expression $\Pi_L(\Pi_M(\dots(\Pi_N(R)))) = \Pi_L(R)$ represents the **Cascade of Projection** rule in relational algebra. Like the cascade of selection, it is a mathematical transformation used by a DBMS to simplify and optimize query execution.

Here is a breakdown of what this expression means and why it is important:

1. Breaking Down the Symbols

- Π (**Pi**): This represents the **projection** operation, which filters columns (attributes) from a table, keeping only those specified in the list and discarding the rest.
- R : This is the **input relation** (the table) being queried.
- L, M, N : These represent **attribute lists** (sets of column names). In a valid sequence, L must be a subset of M , and M must be a subset of N .

Relational Algebra Transformation

(Cascade of Projection)

2. What the Expression States

- **The Left Side ($\Pi_L(\Pi_M(\dots(\Pi_N(R))))$):** This represents a **nested sequence** of projections. It suggests the system should first narrow the table to attributes in list N , then narrow that result further to list M , and finally narrow it to the final required list L .
- **The Right Side ($\Pi_L(R)$):** This represents a **single projection** operation. It tells the system to take the original table R and immediately extract only the final attributes required (list L).
- **The Equals Sign ($=$):** This confirms that both methods are **equivalent**—they will always result in the same set of columns and rows.

Relational Algebra Transformation

(Cascade of Projection)

3. Why This Is Used in Optimization

While the results are the same, applying this transformation helps the DBMS achieve **Heuristic Optimization**:

- **Eliminating Redundant Steps:** The primary purpose of this rule is to simplify the execution plan. Performing multiple nested projections is redundant; the DBMS can save processing time by ignoring all intermediate projection steps and only performing the final one.
- **Reducing Data "Width":** In the larger context of optimization, the DBMS often "**pushes down**" the final projection (L) as close to the data source as possible. By doing this, it discards unnecessary attributes early, which reduces the amount of data (the "width" of the rows) being passed through the rest of the query tree.
- **Improving Memory Efficiency:** By keeping only the attributes needed for the final result, the system minimizes the memory required to store intermediate results during complex operations like Joins or Sorts.

- Relational Algebra Transformation

- The following list gives a basic selection of relational algebra transformation:

- **Commutativity of selection:** A sequence of selection operations are commutative

$$\sigma q(\sigma p(R)) = \sigma p(\sigma q(R))$$

- **Associativity of Natural Join and Cross Product:** Natural join (*) and Cross Product (X) are associative:

$$(R * S) * T = R * (S * T)$$

$$(R X S) X T = R X (S X T)$$

Heuristic Optimization- Further Details

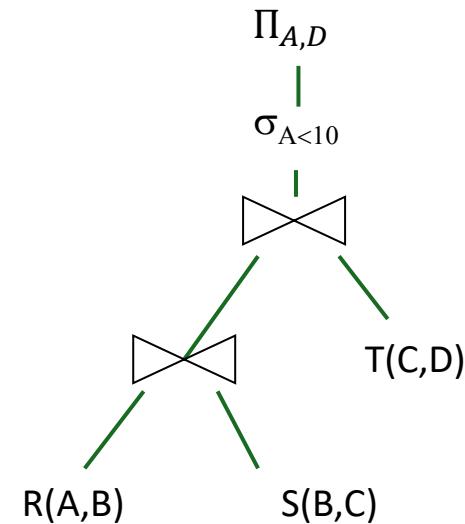
- **Perform selection operations as early as possible**
 - Keep predicates on same relation together
- **Combine Cartesian product with subsequent selection whose predicate represents join condition into a join operation.**
- **Use associative of binary operations to rearrange leaf nodes with most restrictive selection operations first.**
- **Perform projection as early as possible**
 - Keep projection attributes on same relations together.
- **Compute common expressions once.**
 - If common expression appears more than once, and result not too large, store result and reuse it when required.

Given an SQL Query, Let we Translate it into its Relational Algebra Expression

R(A,B) S(B,C) T(C,D)

SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
AND S.C = T.C
AND R.A < 10;

$\Pi_{A,D}(\sigma_{A<10}(T \bowtie (R \bowtie S)))$



Can we Optimize this Query Execution Plan ?

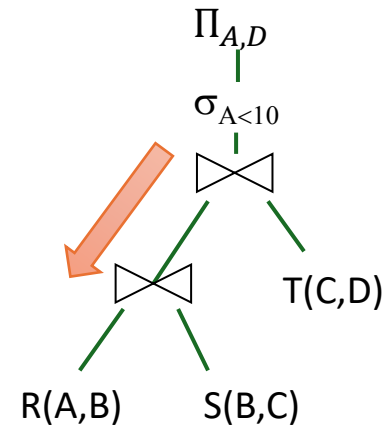
Optimizing RA Plan

R(A,B) S(B,C) T(C,D)

SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
AND S.C = T.C
AND R.A < 10;

Push down selection on A
so it occurs earlier

$\Pi_{A,D}(\sigma_{A<10}(T \bowtie (R \bowtie S)))$



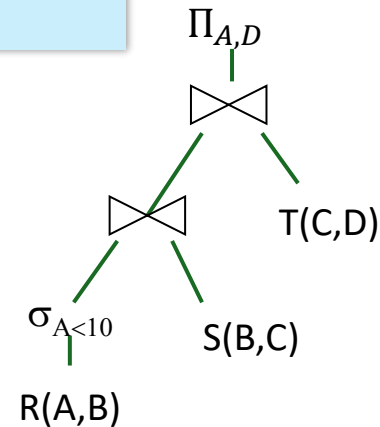
Optimizing RA Plan (cont...)

R(A,B) S(B,C) T(C,D)

SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
AND S.C = T.C
AND R.A < 10;

Push down
selection on A so
it occurs earlier

$\Pi_{A,D}(T \bowtie (\sigma_{A<10}(R) \bowtie S))$



Optimizing RA Plan (cont...)

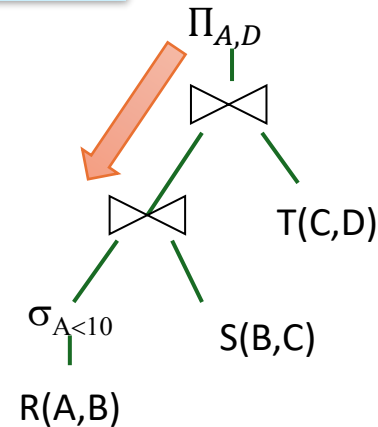
R(A,B) S(B,C) T(C,D)

SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
AND S.C = T.C
AND R.A < 10;



$\Pi_{A,D}(T \bowtie (\sigma_{A < 10}(R) \bowtie S))$

Push down
projection so it
occurs earlier



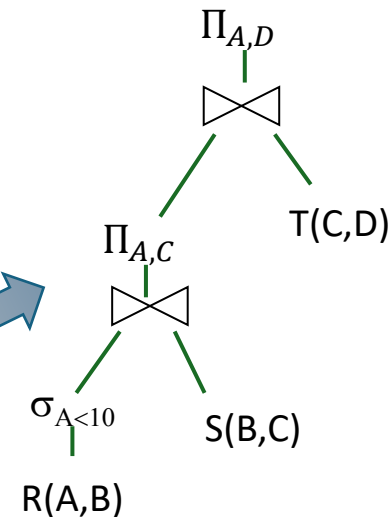
Optimizing RA Plan (cont...)

R(A,B) S(B,C) T(C,D)

SELECT R.A,T.D
FROM R,S,T
WHERE R.B = S.B
AND S.C = T.C
AND R.A < 10;

We eliminate B
earlier!

$\Pi_{A,D} \left(T \bowtie \Pi_{A,C} (\sigma_{A < 10}(R) \bowtie S) \right)$

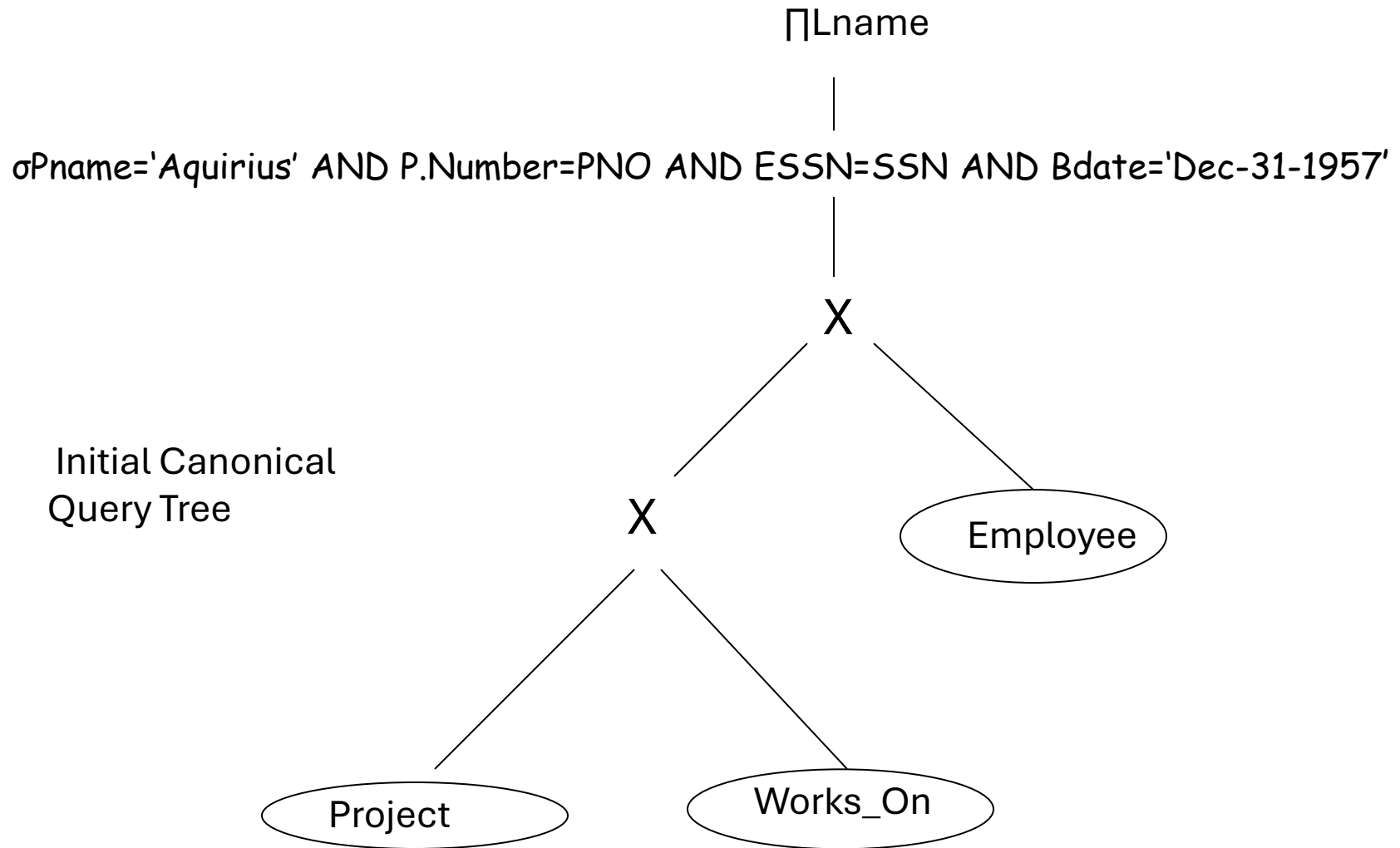


-- Query Optimization: Example ...

- Query: Find the last name of employees born after 1975 who work on a project named 'Aquarius'

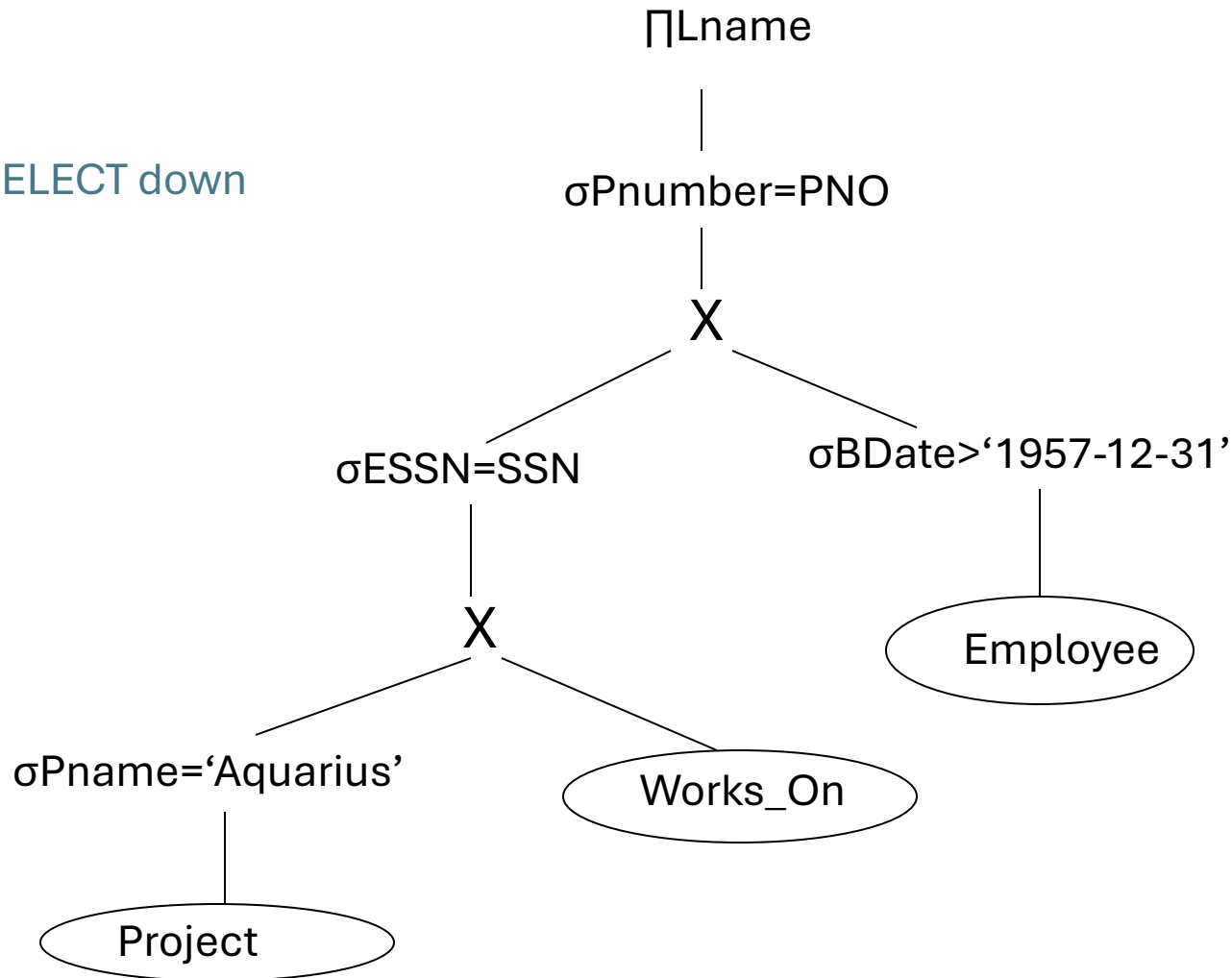
```
SELECT Lname
FROM Employee, Works_on, Project
WHERE Pname = 'Aquarius'
AND ESSN = SSN
AND Pnumber = PNO
AND Bdate > '1975-12-31'
```

... -- Query Optimization: Example ...



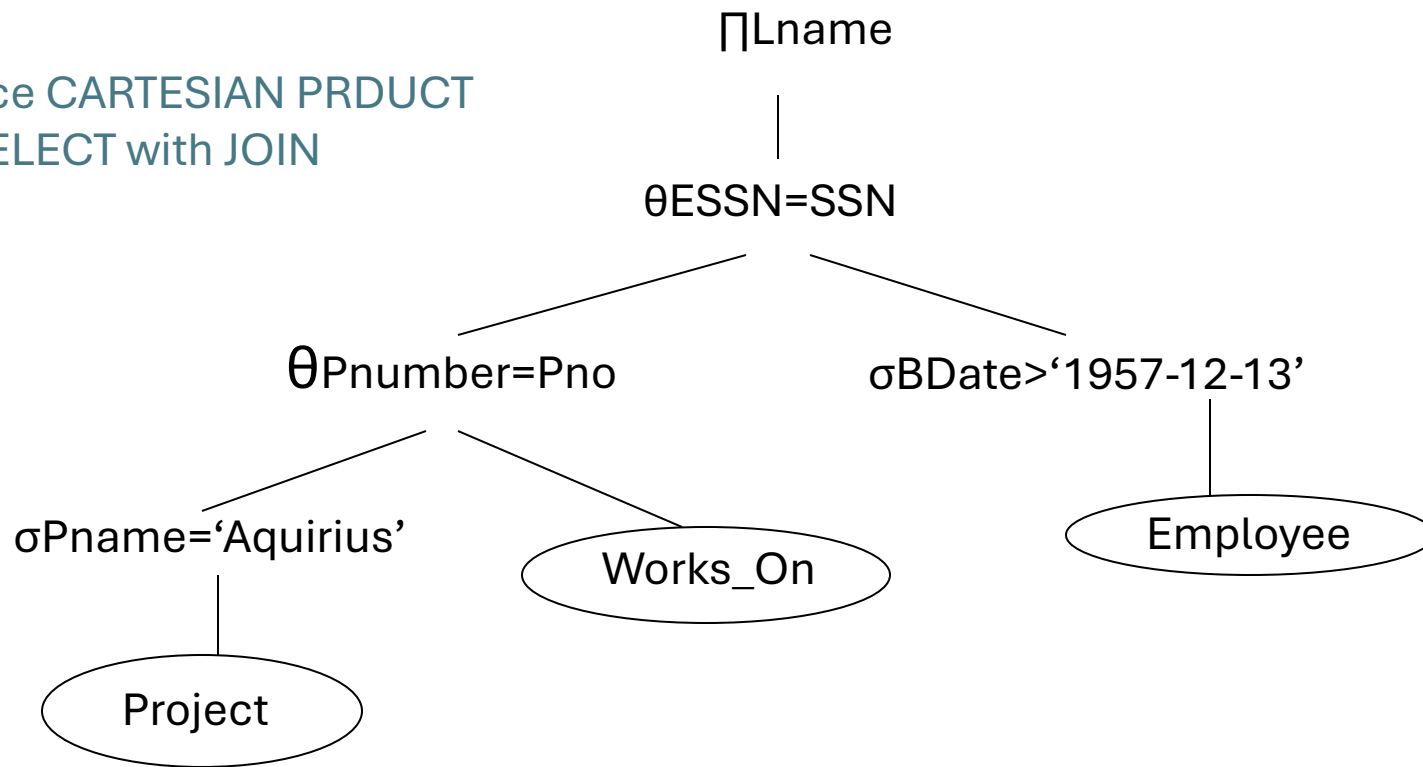
... -- Query Optimization: Example ...

Moving SELECT down



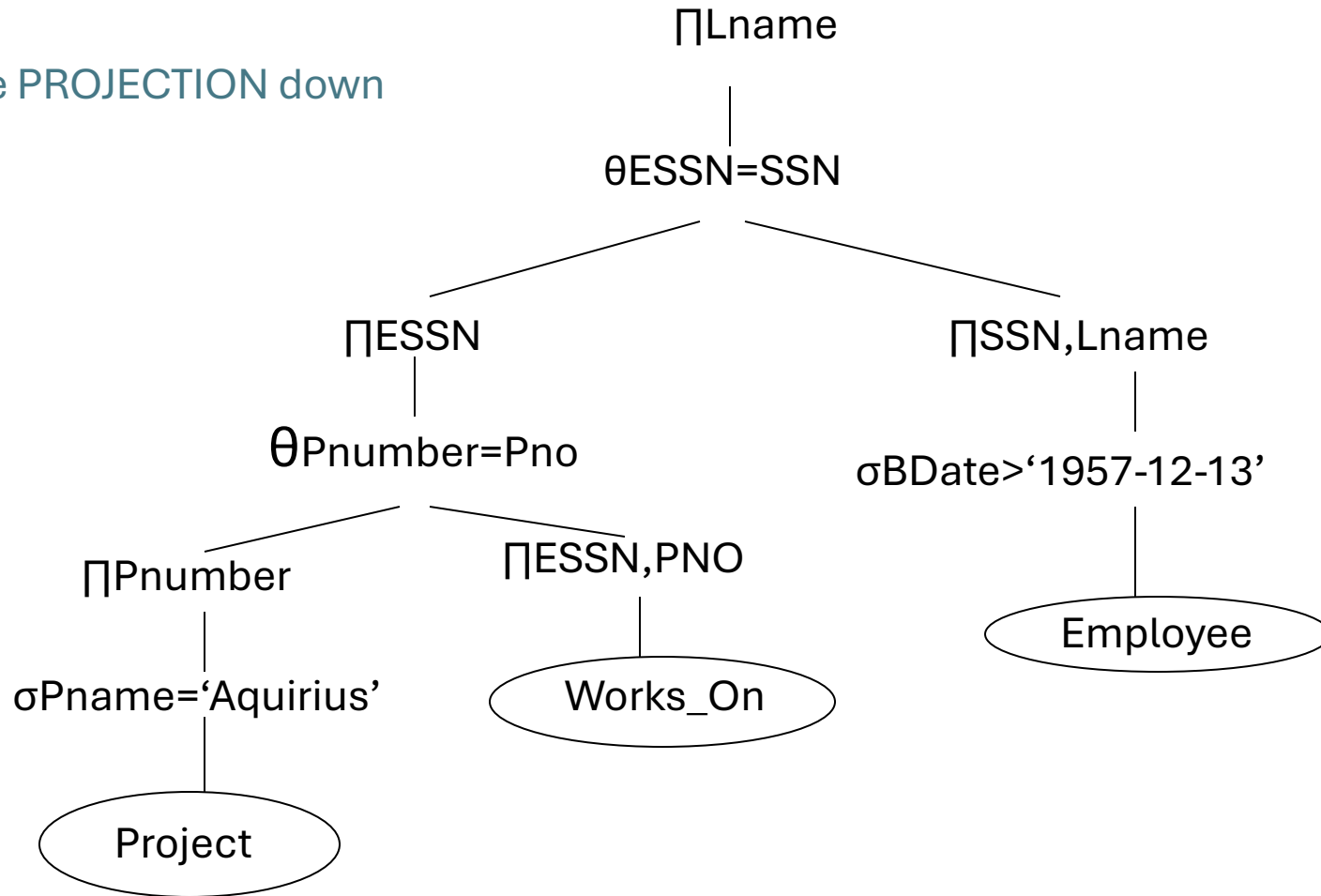
... -- Query Optimization: Example ...

Replace CARTESIAN PRODUCT
And SELECT with JOIN



... -- Query Optimization: Example

Move PROJECTION down



Practice Question

- Find the last names of customers who placed an order for a product named 'Wireless Mouse' after the year 2021.

Customers

customer_id	first_name	last_name	B-year
1	ahmad	khan	1996
2	wahid	jan	2004

order_id	Customer_id	product_id	order-year
201	1	101	2022
202	2	102	2020

Products

product_id	Product_name
101	Wireless Mouse

Any Questions??

